

E0245 HPCA Assignment 1

Implementing the Hawkeye Cache Replacement Policy in ChampSim

Anirudh Gupta, Sr. No: 23600

September 15, 2026

1 Introduction

The goal of this assignment is to implement the Cache Replacement Policy(CRP) - Hawkeye from **Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement**, with ChampSim simulator and evaluate its performance by measuring Cache Miss Rates(CMR) against the default LRU policy. We test the performance against 3 different associativities and 3 different datasets too. Beyond these I experimented around with different parameters and ideas, to see whether the chosen method/parameters are actually well chosen or not. These experiments are just to satisfy some curiosity. and patterns in data. I will be mentioning some of them below while discussing the my learnings.

GitHub Repository Link: <https://github.com/AnirudhG07/Champsim>, contains the code for the implementation of Hawkeye cache replacement, result files and plots inside `experiments/` directory with a README mentioning steps to reproduce the results.

2 Learnings/Observations

Top 3 learnings/observations while implementing this assignment are mentioned below.

2.1 Training Improve CMR

LRU CRP even though widely used, shows lesser performance than the Hawkeye CRP. This is a not just a Hawkeye, but a general observation from other papers in different CS fields too, that any fixed heuristic based, static parameters based, and a non-learning based approach will always be outperformed by a learning based approach. That latter might need more time, resources, and data to train, but it usually outperforms the former, sometimes by even a significant margin. This is visible in our results too.

Concretely, Hawkeye replaces a fixed heuristic with a feedback loop. OPTgen reconstructs what Belady's optimal algorithm would have done over the recent access stream, which yields a ground-truth label for every reference: was this line worth keeping? That label is attributed back to the load instruction that brought the line in, and accumulated in a saturating counter indexed by that PC. At fill time the policy no longer guesses from recency alone — it asks what this particular instruction's lines have historically been worth. LRU, by contrast, applies one rule to every line regardless of which instruction requested it, so a streaming load that never reuses its data is treated identically to a hot pointer dereference. The gain comes from exactly that distinction, which is why it is largest on workloads that mix access patterns and smallest on workloads that are uniformly streaming.

2.2 Implementation based Learnings

Here I talk about some insights and observations I made while implementing the Hawkeye CRP in the ChampSim simulator.

2.2.1 Reusing the access sequence table

Both OPTgen and the Hawkeye adapter need a per-set history of the last $8W$ accesses, but for different reasons. OPTgen stores the *block address* accessed at each step, and scans backwards for a matching address to recover the usage interval; it never sees a PC, since its interface is `access(set_idx, address)`. The adapter needs the *PC* of the previous access to the same block, because OPTgen’s verdict grades the access that loaded the line, not the one demanding it now.

My first implementation kept a second ring in `hawkeye.cc` holding both the block address and the PC, and scanned it independently — duplicating OPTgen’s address array and repeating the same backward walk twice per access. Since both rings are indexed by the same clock and cover the same window, this is redundant. Exposing two read-only accessors from OPTgen, `current_time(set)` and `last_distance(set)`, lets the adapter drop its address array entirely and keep only `pc_seq`, indexed by OPTgen’s clock. This removes roughly 2MB of state, halves the per-access scanning, and makes clock drift between the two structures impossible. I verified the refactor is behaviour-preserving: the LLC miss rate was identical on both benchmarks before and after.

2.2.2 Significance of `access_type::WRITE` in CRP

Apart from just the logic of Hawkeye, champsim other CRP’s used `access_type::WRITE` to deal with writebacks. That depends on the CRP and is different for different CRP’s as it can be seen in `lru.cc`, `ship.cc`. **What is `access_type::WRITE`?** While scowering through the definitions, I found that `access_type::WRITE` is not associated with any ip at all, rather it is used to mark a block evicted from higher level cache and written to lower level(LLC). Since we deal with LLC, we need to handle this case in our CRP too.

In the function `update_replacement_state()`, we deal with a general behavior of the CRP, which for us means predict, train and update rrpv. For a cache block, if it is a writeback, we can’t need to train the predictor since it is not associated with any ip. We also avoid searching through the access sequence history table, because that lookup exists only to recover the PC to train with, and a writeback has none — `cache.cc` constructs the packet with `writeback_packet.ip = champsim::address{}`. Feeding a writeback to OPTgen would in fact be worse than useless: it is not a demand reference, yet it would occupy a slot in the occupancy vector and shift every later timestamp, distorting the usage intervals of the real accesses around it. Though for such a line, since it is a `FILL` in the cache block, the `replacement_cache_fill` function deals with it and we still need to update the rrpv of the block since we know the `set`, `way` for the block, and we can update the rrpv of the block between 0-7 or not set. However, since the block will replace an evicted line, it will be put in the cache with a rrpv having 7, which is equivalent to not setting the rrpv of the block. Logically it makes sense to keep it at 7 since the line since it is a writeback, it might not cache friendly and hence better of not to cache it. If it sees a hit on itself, we will update the rrpv of the block to 0, and it will be future tracked by the whole pipeline.

I tried setting the rrpv of the block to 0-7 and not-setting at all, and the results are as expected. It is better of not setting the rrpv of the block than marking it with a rrpv of 0-6 for potential usage in the future, this is confirmed in the experiments section 4.1 in Figure 3, where we see that miss rates is linearly decreasing from 0-7 and CMR of 7 and none(not setting) is exactly same.

Hence dealing with `access_type::WRITE` is important in the CRP, and we need to handle it properly.

P.S. Coding this assignment by digging into different files like `cache.cc`, `lru.cc`, etc. made me realize that implementing a CRP is less complex than I thought. I always imagined it to be a 1000 lines of code, but it is quite cute, especially `lru`. Even though it is just a simulator, but still real world systems won't be that complicated either, hopefully.

2.3 Result Observations

2.3.1 Improvements of CMR with Associativity

As the associativity of the cache increases, the CMR decreases. This is because with more ways, there are more blocks in each set. However notice that the reduction in CMR is not linear with associativity. This trend makes sense since associativity only removes *conflict* misses, and those are largely exhausted by eight ways. Going from four to eight ways gives each set twice the room to hold competing lines, which eliminates most of the conflicts that a 4-way set cannot absorb. Beyond that, the misses that remain are capacity and compulsory misses, which more ways cannot fix — the total LLC capacity is held fixed at 2,MB across the sweep, so increasing associativity only redistributes the same number of blocks into fewer, larger sets. The curve therefore flattens, and the benefit of a smarter replacement policy shrinks with it: with 16 candidates per set, even LRU usually has a reasonable line to evict, whereas at four ways a single bad eviction is far more costly.

2.3.2 Observation of CRP on different datasets

Across the three benchmarks the outcome tracks how much reuse there is to exploit. `456.hmmmer` and `473.astar` improve by 2.79% and 1.56%, while `429.mcf` regresses by 11.32%. The difference is based on workloads in different datasets. On `mcf` only 37% of LLC references find a previous access to the same block inside OPTgen's 8W history window, against 60% on `hmmmer` and 78% on `astar` (these results are self generated as explained in the Experiments section 4.3). The predictor is therefore trained on far fewer samples, and its decisions are less reliable. The result is that Hawkeye's informed guesses are worse than LRU's uninformed ones, and the miss rate rises for `mcf`. This shows that the different workloads have different behavior and a CRP should be able to minimize the CRP overall on such workloads.

3 Results

3.1 LLC Miss Rate versus Associativity

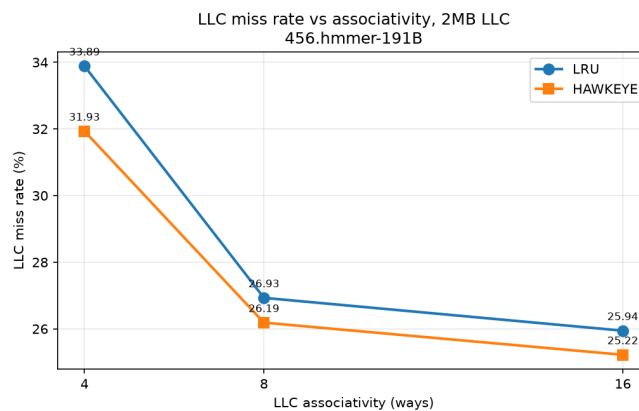


Figure 1: LLC miss rate versus LLC associativity on `456.hmmmer-191B`, with total LLC capacity held fixed at 2 MB (64 B lines) by scaling the set count inversely. 20 M warmup / 50 M simulated instructions. Lower is better.

Ways	Sets	LRU (%)	Hawkeye (%)	Reduction (%)
4	8192	33.89	31.93	5.77
8	4096	26.93	26.19	2.76
16	2048	25.94	25.22	2.79

Table 1: LLC miss rate versus associativity on 456.hmmmer-191B.

Both policies improve sharply from four to eight ways and then flatten, confirming that conflict misses dominate at low associativity and are largely exhausted by eight ways. More interesting is that Hawkeye’s *advantage* over LRU is greatest where the cache is most constrained: 5.77% at four ways, against roughly 2.8% at eight and sixteen. With only four candidates per set a single poorly chosen eviction is expensive and there is little slack to recover from it, so an informed decision is worth more. At sixteen ways LRU almost always has a reasonable victim available on recency alone, and the margin for a smarter policy narrows correspondingly. That the figures at eight and sixteen ways are nearly identical suggests the misses which remain are capacity and compulsory misses, which no replacement policy can remove.

3.2 Miss-Rate Reduction across Benchmarks

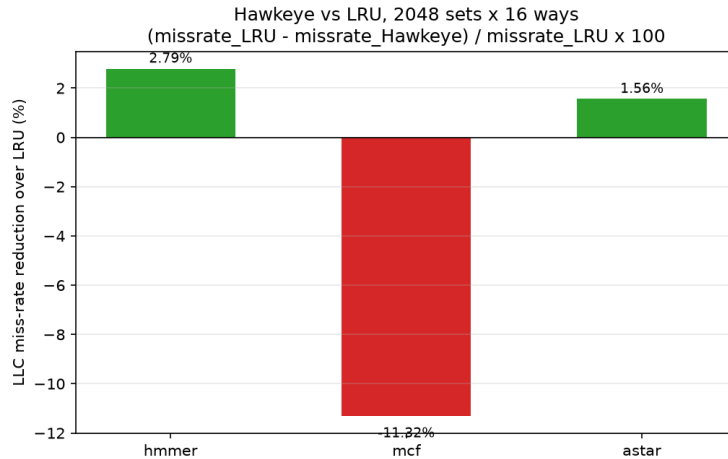


Figure 2: LLC miss-rate reduction over LRU for Hawkeye across three SPEC CPU2006 benchmarks at the default LLC geometry (2048 sets $\times$ 16 ways), computed as $(\text{missrate}_{\text{LRU}} - \text{missrate}_{\text{Hawkeye}}) / \text{missrate}_{\text{LRU}} \times 100$. Positive is better.

Benchmark	LRU (%)	Hawkeye (%)	Reduction (%)
456.hmmmer-191B	25.94	25.22	2.79
429.mcf-22B	64.82	72.16	-11.32
473.astar-42B	19.63	19.33	1.56

Table 2: LLC miss rate at the default geometry (2048 sets, 16 ways).

4 Experiments

This section contains some experiments I did to see the effect of different parameters on the performance of the Hawkeye CRP and why.

4.1 Effect of the Writeback Insertion RRPV

While implementing the Hawkeye CRP, usage of `access_type::WRITE` is important to handle the writeback fills in the cache as shown in Figure 1. As discussed in section 2.2.2, these writeback fills are not associated with any ip, but they still need to be updated in the `rrpv` (we have the block and set value). By logic mentioned previously, best value would be to have it at 7 or not set at all (should be by equivalent by construction). Hence to verify which value of `rrpv` is best, I did an experiment where I set the `rrpv` of the writeback fill to 0-7 and not set at all, and measured the miss rate reduction over LRU. The results are shown in Figure 3 and Table 3.

Listing 1: Handling of `access_type::WRITE` in `update_replacement_state()`.

```
if (type == ACCESS_TYPE_WRITE) { /* for writeback hits */  
    /* rrpv[set][way] = value; */ /* choose the appropriate RRPV value */  
    return;  
}
```

The results show that the best value of `rrpv` for writeback fills is 7 or not setting it at all, which is as expected.

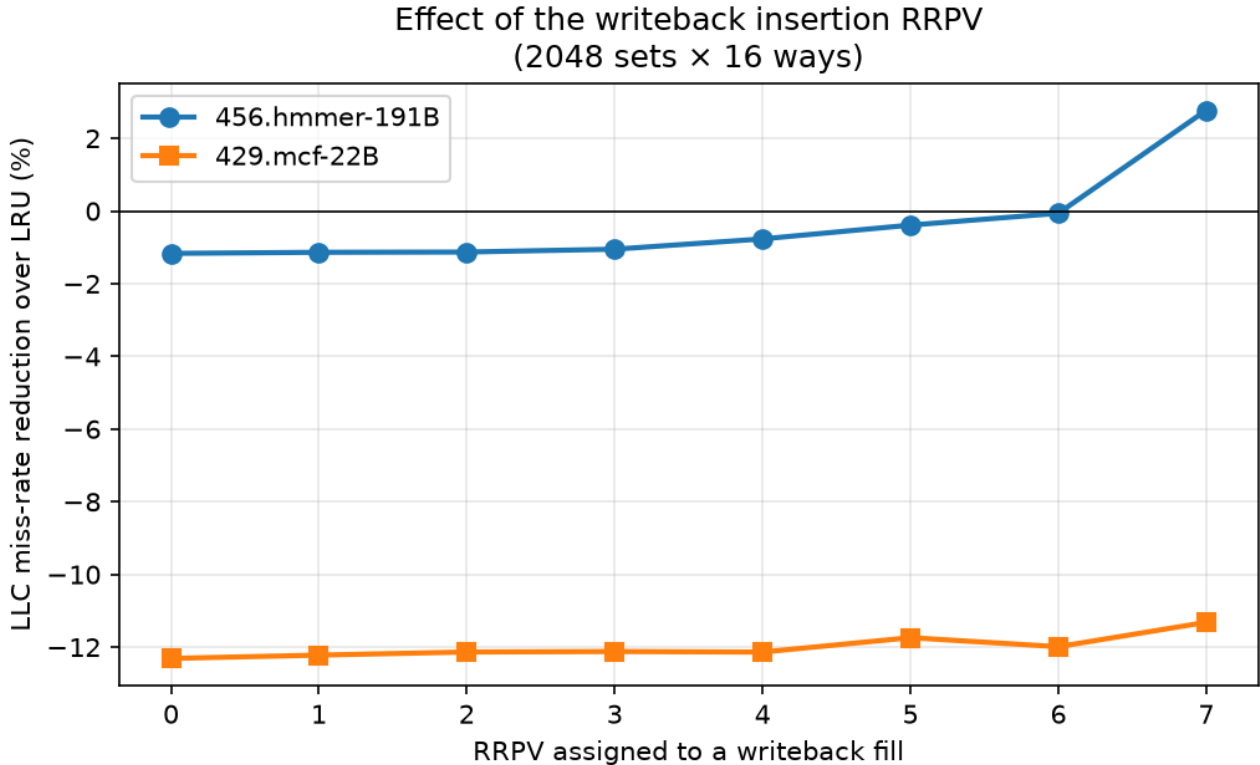


Figure 3: LLC miss-rate reduction over LRU as a function of the RRPV assigned to a writeback fill in `replacement_cache_fill`, for both `find_victim` variants. `none` denotes not setting the RRPV at all and letting the predictor classify the writeback using its (absent) ip. Measured at the default LLC configs.

Writeback RRPV	456.hmmmer-191B	429.mcf-22B
0	-1.17	-12.31
1	-1.14	-12.22
2	-1.13	-12.14
3	-1.05	-12.12
4	-0.77	-12.14
5	-0.39	-11.74
6	-0.06	-11.99
7	2.77	-11.32
none	2.77	-11.32

Table 3: LLC miss-rate reduction over LRU (%) for each writeback insertion RRPV, using the assignment’s `find_victim`. Higher is better. The `none` row — not setting the RRPV at all — is numerically identical to RRPV 7, and is omitted from Figure 3 for clarity.

4.2 RRPV update, Paper vs Assignment

The assignment mentions that, while finding the victim using the `rrpv`, we need to age all lines until we find a line with `rrpv=7`(or if previously present). Meanwhile the paper mentions that finding the value with maximum `rrpv` and evicting it is enough. The paper also mentions that a cache friendly line should never have `rrpv = 7`. Comparing the two methods theoretically, the paper’s method should be better since it is not letting multiple cache friendly lines to be aged to 7. Once we start aging all lines, the victim will remain the same for that instruction, but for the future victims, we promotd a cache friendly line to 7 which would be evicted in the future. Hence the paper’s method avoids such increments and lets only cache averse lines be evicted primarily, and only unless they donot exist that we evict a cache friendly line.

To test this hypothesis I made the two methods within `rrpv`’s `find_victim()` function and measured the miss rate reduction with each other. Figure 5 shows the two methods. The results, as observable in Figure 4, verify the hypothesis on `456.hmmmer` but not on `429.mcf`.

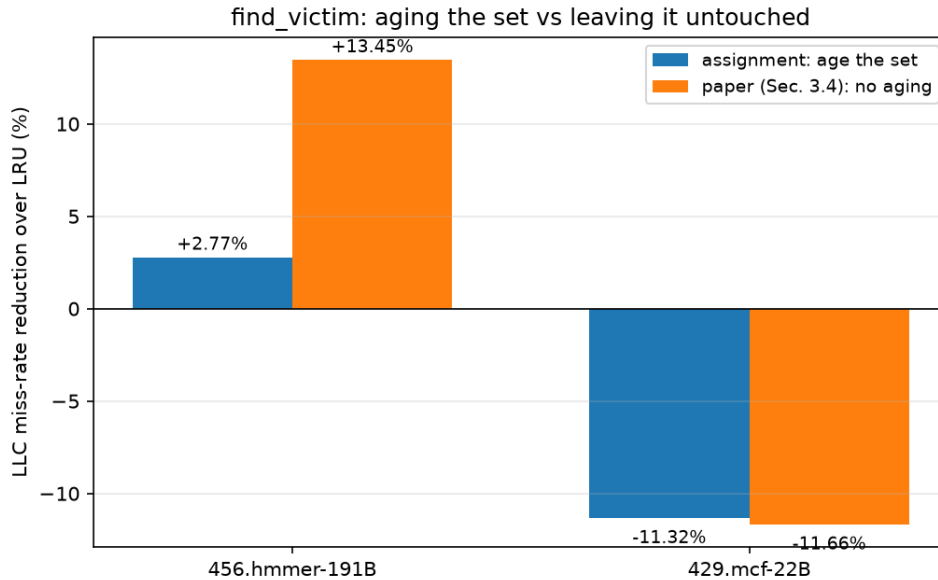


Figure 4: LLC miss-rate reduction over LRU for the two `find_victim` variants of Figure ??, at the default LLC geometry (2048 sets $\times$ 16 ways). Removing the aging step is worth almost five times the reduction on `456.hmmmer`, but makes no useful difference on `429.mcf`.

Assignment: age the set

```
// find the way holding the maximum RRPV
int max_rrpv = rrpv[0];
size_t victim = 0;
for (size_t i = 1; i < rrpv.size(); i++) {
    if (rrpv[i] > max_rrpv) {
        max_rrpv = rrpv[i];
        victim = i;
    }
}

// age every line until the max reaches 7
const int distance = 7 - max_rrpv;
if (distance > 0) {
    for (size_t i = 0; i < rrpv.size(); i++) {
        rrpv[i] += distance;
    }
}

return victim;
```

Paper (Sec. 3.4): no aging

```
// find the way holding the maximum RRPV
int max_rrpv = rrpv[0];
size_t victim = 0;
for (size_t i = 1; i < rrpv.size(); i++) {
    if (rrpv[i] > max_rrpv) {
        max_rrpv = rrpv[i];
        victim = i;
    }
}

// the set is left untouched

return victim;
```

Figure 5: The two `find_victim()` variants, showing only the part that differs. Both walk the RRPV vector once and return the way holding the maximum, so they always select the *same* victim; they differ solely in whether the set is aged as a side effect. The assignment’s rule raises every RRPV by $7 - \text{max}$, which promotes cache-friendly lines to 7 and makes them indistinguishable from cache-averse ones on subsequent evictions.

4.3 Understanding Dataset Patterns

429.mcf was the one benchmark on which Hawkeye lost to LRU, and none of the design choices swept above accounted for it. To find out why, I temporarily added five counters to `struct hawkeye` and printed them once at the end of the run through `ChampSim’s replacement_final_stats()` hook. Only the added lines are shown; they were removed again before submission.

Listing 2: Counters added to `hawkeye` (additions only).

```
// members of struct hawkeye
long diag_accesses = 0, diag_opt_hits = 0, diag_prev_found = 0;
long diag_fills = 0, diag_fills_friendly = 0;

// in update_replacement_state(), after d = optgen.last_distance(set)
diag_accesses += 1;
diag_opt_hits += optgen_hit ? 1 : 0;
diag_prev_found += (d > 0) ? 1 : 0;

// in replacement_cache_fill(), after the prediction is made
diag_fills += 1;
diag_fills_friendly += _pred ? 1 : 0;

// ChampSim calls this once, after the simulation ends
void hawkeye::replacement_final_stats() {
    auto pct = [](long a, long b) { return b ? 100.0 * a / b : 0.0; };
    std::cout << "HAWKEYE-DIAG"
        << " opt_hit_rate=" << pct(diag_opt_hits, diag_accesses)
        << " prev_found_rate=" << pct(diag_prev_found, diag_accesses)
        << " friendly_fill_rate=" << pct(diag_fills_friendly, diag_fills) << "\n";
}
```

Benchmark	OPT hit (%)	Prev. found (%)	Friendly fills (%)	Reduction (%)
473.astar-42B	78.36	78.60	99.99	1.56
456.hmmmer-191B	58.22	60.49	97.75	2.79
429.mcf-22B	24.93	37.47	75.08	-11.32

Table 4: Policy-internal rates at the default LLC geometry. *Prev. found* is the fraction of references having an earlier access to the same block inside OPTgen’s 8*W* history window.

The three benchmarks order identically by *prev. found* and by outcome. On **astar** and **hmmmer** the OPT hit rate almost equals the prev. found rate (78.36 against 78.60, and 58.22 against 60.49), meaning that whenever a previous access lies inside the window it is nearly always an OPT hit — OPTgen is answering from evidence. On **mcf** only 37% of references have a previous access at all, and of those barely two thirds are hits, so the majority of references are labelled OPT misses by default rather than by observation. Those labels train the corresponding PCs negatively, the predictor classifies their lines cache-averse, and they are inserted at RRPV 7 and evicted almost immediately — lines that LRU would have kept. **mcf** streams through a working set far larger than the 2 MB LLC, so there is little reuse for any policy to capture, while Hawkeye still pays the cost of every misprediction. The regression is therefore a property of the access stream meeting a finite history window, not an implementation error.
